

Digital Hibernation: A Decision-Theoretic Framework for Asymmetric-Cost Idle-State Selection in Modern Processors

Abdullah Hawas
Independent Researcher
Dhi Qar, Iraq
abdullahhawas93@gmail.com

July 2026

Abstract

This paper revisits the problem of idle-state selection in modern processors through the lens of decision theory. We demonstrate that existing point-estimate governors, while computationally efficient, fail to account for the fundamentally asymmetric nature of prediction errors: choosing a shallow state for a long idle wastes energy, while choosing a deep state for a short idle incurs transition penalties without benefit.

We propose Expected-Cost, a lightweight governor that minimizes expected energy over the empirical distribution of recent idle durations. Using 10 independent runs on Intel Core i7-7xxx hardware with RAPL measurements, we show that Expected-Cost achieves 28.57% energy savings (418.67 ± 27.21 J) compared to the default menu governor (586.14 ± 38.10 J), with statistical significance ($p < 0.00001$).

We further report a counter-intuitive negative result: a tabular Q-learning agent, despite learning effectively in simulation, fails to outperform the analytic rule on real hardware. This finding establishes practical boundaries for reinforcement learning in kernel-level power management. All code and traces are publicly available for reproduction.

1 Introduction

The problem of idle-state selection (C-State selection) remains one of the most persistent challenges in processor power management. Modern processors expose a hierarchical ladder of sleep states. Deeper states reduce power consumption but incur higher transition latencies and energy costs. Selecting the optimal state requires predicting the upcoming idle duration, a task complicated by the fact that the true duration is unknown at decision time.

Existing Linux governors—menu and teo—approach this problem through point estimation. Each governor computes a single predicted idle duration and selects the deepest state whose minimum residency that prediction satisfies. This approach makes an implicit but critical assumption: that over-prediction and under-prediction errors carry equal cost. We demonstrate that this assumption is false.

1.1 The Asymmetric Cost Structure

The two error types have fundamentally different consequences. Under-prediction (choosing a shallow state for a long idle) wastes power over the entire idle interval. Over-prediction (choosing a deep state for a short idle) pays the full transition energy without realizing any significant residency benefit. The wake penalty dominates the cost structure, making over-prediction disproportionately expensive in many workloads.

This asymmetry suggests that the optimal decision rule should minimize expected cost over the distribution of possible idle durations, rather than making decisions based on a single point estimate. We formalize this insight and validate it on real hardware.

1.2 Contributions

This paper makes four contributions:

1. We formalize C-State selection as a decision problem under asymmetric cost, deriving the Expected-Cost rule that minimizes expected energy over the empirical distribution of recent idle durations.
2. We implement Expected-Cost as a proof-of-concept cpuidle governor and evaluate it on real Intel hardware using RAPL measurements. Our results show 28.57% energy savings over the menu governor.
3. We train a tabular Q-learning agent on the same workload and demonstrate that, despite learning in simulation, it fails to outperform the analytic rule on real hardware.
4. We identify a fundamental irreducibility bound: any policy that must decide before observing the true idle duration faces inherent regret proportional to the variance of idle durations.

2 Background

2.1 C-State Ladder

Intel’s ACPI specification defines a hierarchy of idle states. Table 1 summarizes the parameters used in our energy model.

Table 1: C-State parameters for Intel Core i7-7xxx (Kaby Lake)

State	Power (W)	Min Residency	Trans. Energy	Description
POLL	15.0	0	~ 0 nJ	Active polling
C1	10.0	1 μ s	0 μ J	HLT instruction
C1E	5.0	10 μ s	1 μ J	Enhanced halt
C3	2.0	80 μ s	2 μ J	Cache flush
C6	0.5	200 μ s	20 μ J	Core power-gate
C8	0.1	500 μ s	100 μ J	PLL off
C10	0.05	1000 μ s	500 μ J	Platform sleep

2.2 Decision Under Asymmetric Cost

The cost of a prediction error in C-State selection is inherently asymmetric. Let t be the true idle duration, $\hat{t}$ the predicted duration, and $s(t)$ the selected state. The cost functions are:

$$C_{\text{under}}(t) = (P_{\text{shallow}} - P_{\text{deep}}) \cdot t$$

$$C_{\text{over}} = E_{\text{trans}}$$

Under-prediction cost scales with t , while over-prediction cost is fixed and dominated by transition energy. This asymmetry necessitates a distribution-aware decision rule. The Expected-Cost policy selects the state minimizing:

$$\mathbb{E}[E(t, s)] = \frac{1}{n} \sum_{i=1}^n [P_s \cdot t_i + E_{\text{trans}}(s) + \mathbf{1}(t_i < T_{\text{res}}(s)) \cdot 3E_{\text{trans}}(s)]$$

This formulation accounts for both error types without requiring explicit estimation of the cost ratio.

3 Methodology

3.1 Energy Model

Energy consumption for an idle interval of duration t in state s is:

$$E(t, s) = P_s \cdot t + E_{\text{trans}}(s) + \text{Penalty}(t < T_{\text{res}}(s))$$

where $\text{Penalty} = 3 \times E_{\text{trans}}(s)$ when the idle ends before the state’s minimum residency target.

3.2 Policies Compared

We compare four policies:

1. **Threshold (menu)**: Selects the deepest state whose minimum residency is satisfied by the median of recent idle durations.
2. **Expected-Cost**: Selects the state that minimizes expected energy over the empirical distribution of recent idle durations, with a window size of 16.
3. **RL Agent**: A tabular Q-learning agent trained on the same traces, with state encoding based on distribution shape and trend, $\gamma = 0.7$, $\alpha = 0.1$.
4. **Oracle**: A theoretical baseline that knows the true idle duration before selecting a state.

Performance is measured using Energy Regret:

$$\text{Regret} = \frac{E_{\pi} - E_{\text{Oracle}}}{E_{\text{Oracle}}} \times 100\%$$

3.3 Hardware Setup

All experiments were conducted on an Intel Core i7-7xxx (Kaby Lake) processor running Linux Mint 21.x (kernel 6.x). Energy measurements use Intel RAPL via:

```
perf stat -e power/energy-pkg/ -a
```

Idle traces were captured using trace-cmd and extracted per-core. We used a realistic bursty workload: 5,000 iterations of sha512sum interleaved with random sleeps (1-9 ms).

3.4 Statistical Protocol

Each experiment was repeated 10 times. For each run, we compute total energy consumption over a 30-second workload. Statistical significance was assessed using a two-tailed paired t-test with $\alpha = 0.05$. Confidence intervals (95%) are reported alongside means and standard deviations.

4 Results

4.1 Simulation Results

4.1.1 Multi-State Ladder Performance

Table 2 summarizes simulation results on the regime-switching workload.

Table 2: Simulation results on regime-switching workload			
Policy	Energy Regret	Oracle-Hit Accuracy	Mean Rung Error
Threshold	411.0%	35.7%	0.95
Expected-Cost	47.6%	30.0%	1.13

The Expected-Cost policy achieves substantially lower energy regret (47.6% vs 411.0%) despite lower exact-state accuracy (30.0% vs 35.7%). This confirms that accuracy is a misleading metric under asymmetric cost.

4.1.2 RL Agent Performance

The Q-learning agent was trained for 500,000 episodes. Figure 1 illustrates the learning curve.

Table 3: RL agent final regret after 500K episodes

Policy	Final Regret
Threshold (menu)	130.6%
Expected-Cost	55.3%
RL (start)	192%
RL (end)	88.9%

The agent improves from 192% to 88.9% regret, approaching but not surpassing the Expected-Cost baseline.

4.2 Hardware Validation Results

Table 3 summarizes energy measurements across 10 independent runs.

Table 4: Energy consumption across policies (10 runs)			
Policy	Mean Energy (J)	Std Dev (J)	95% CI
menu	586.14	38.10	[558.88, 613.39]
teo	539.89	5.58	[535.90, 543.88]
Expected-Cost	418.67	27.21	[399.21, 438.14]

Expected-Cost achieves 28.57% energy savings compared to menu. Statistical significance was confirmed using a paired t-test:

- menu vs Expected-Cost: $t = 11.31$, $p < 0.00001$
- menu vs teo: $t = 3.80$, $p = 0.00132$

Figure 2 presents a boxplot of the energy distributions.

4.3 Window Size Sensitivity

Table 5: Expected-Cost sensitivity to window size		
Window Size	Regret (Threshold)	Regret (Expected-Cost)
8	58.8%	39.4%
16	68.4%	42.0%
32	89.1%	45.7%
64	128.4%	54.6%
128	211.8%	71.9%

Small windows (8-16) outperform large windows (128-256) in regime-switching environments. This contradicts the assumption that more data always improves decisions.

4.4 RL Failure on Real Hardware

Despite learning in simulation, the Q-learning agent underperforms Expected-Cost on real hardware:

Table 6: RL vs Expected-Cost on hardware			
Policy	Regret	Energy (J)	Savings vs menu
Threshold	103.32%	677.85	—
Expected-Cost	45.23%	484.18	28.6%
RL Agent	56.00%	520.09	23.3%

The agent fails due to three factors:

1. Interrupt noise destroys the regime-switching structure RL exploits.

2. ϵ -greedy exploration selects suboptimal states that waste real energy.
3. Distribution shift between training traces and test conditions.

5 Implementation Roadmap

5.1 Proof-of-Concept Kernel Module

We developed a skeletal C implementation of the Expected-Cost governor. The PoC translates distributional logic into the `cpuidle_governor API structure`. *Key adaptations required for mainline inclusion include*

A lock-free ring buffer for safe multi-core updates.

Integration with tickless (NOHZ) infrastructure.

Proper handling of RCU and preemption contexts.

The complete PoC code is available in the supplementary repository.

5.2 Multi-Architecture Extension

The mathematical framework is architecture-agnostic. Porting to AMD requires updating residency and power parameters to match CC-state specifications. ARM architectures present a greater challenge: idle states are managed via PSCI firmware calls rather than ACPI C-states, and ARM processors lack RAPL counters. Energy validation on ARM requires external instrumentation (e.g., INA219 current sensors).

5.3 Upstream Submission Strategy

We do not propose direct mainline submission. Instead, we intend to first offer this as an out-of-tree module for edge-server use-cases. A subsequent RFC to the Linux PM mailing list would follow after community feedback.

6 Discussion

6.1 Why RL Fails

The RL agent’s failure is not an algorithmic deficiency; it is a positive result that defines the boundary of RL applicability. In i.i.d. environments, Expected-Cost reaches the distributional optimum directly. RL converges slower and with residual regret. RL wins only when the environment contains temporal structure a memoryless rule cannot exploit. This structure is destroyed by real-world interrupt noise.

6.2 The Irreducibility Bound

All realistic policies maintain a gap to the Oracle because Oracle sees the true idle duration before deciding. Any policy that must decide before observing the outcome faces irreducible regret proportional to the variance of idle durations within each regime. This bound is a consequence of stochasticity in the environment, not a failure of the policy.

6.3 Why Periodicity Fails

A recency window of 16–64 idle durations captures slow daily patterns equally well; periodicity adds no incremental information. This negative result rules out unnecessary complexity and directs effort toward the true source of performance: asymmetric cost.

7 Related Work

The Linux kernel ships two primary governors: `menu` and `teo`. Both use point-estimate threshold policies. Our contribution formalizes the cost structure these governors implicitly assume and demonstrates its suboptimality under asymmetric costs.

Reinforcement learning has been applied to DVFS, mobile power management, and data-center scheduling. This work is distinguished by targeting per-idle C-State selection specifically and honestly reporting RL failure on real hardware.

8 Limitations and Future Work

1. This study relies on a single Intel Kaby Lake processor. Results may vary across microarchitectures.
2. The Expected-Cost governor has not yet been benchmarked as a custom `cpuidle` governor on bare-metal Linux.
3. We did not test multi-core processors where core-parking decisions interact.
4. Absolute energy numbers are indicative; relative comparisons and structural conclusions are robust to parameter variation.

Future work includes:

1. Testing on AMD Zen and ARM architectures (A76, N1).
2. Developing a production-ready kernel module with proper locking.
3. Submitting an RFC to the LKML for community review.

9 Conclusion

This paper presented a complete framework for processor idle-state selection grounded in decision theory and reinforcement learning. Three principal findings emerged:

1. The Expected-Cost policy reduces energy consumption by 28.57% compared to the default menu governor on real Intel hardware ($p < 0.00001$).
2. A Q-learning agent learns from reward alone but fails to surpass the analytic rule on real hardware, defining practical boundaries for RL in kernel-level power management.
3. RL advantage is conditional on temporal structure in idle patterns, destroyed by real-world interrupt noise.

The methodology demonstrates the value of controlled experimentation in systems research: each hypothesis was tested with an isolated variable and a negative control. The negative results are as informative as the positive ones.

Data Availability

All simulation code, hardware traces, analysis scripts, and the Expected-Cost kernel PoC are available at: <https://github.com/abdullahhawas/digital-hibernation>

References

- [1] Pallipadi, V. and Starikovskiy, A. (2006). The cpuidle subsystem. *Linux Symposium*, Ottawa, vol. 2, pp. 341–353.
- [2] Elkan, C. (2001). The Foundations of Cost-Sensitive Learning. *Proc. IJCAI*, 973–978.
- [3] Watkins, C. and Dayan, P. (1992). Q-Learning. *Machine Learning*, 8(3-4), 279–292.
- [4] Sutton, R. and Barto, A. (2018). *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition.
- [5] Intel Corporation (2023). Intel 64 and IA-32 Optimization Reference Manual. Document #248966-046.